Design patterns and backend architecture

Chapter 3: Object-level design patterns

Practice designs, not claims about an employer.

Code examples are illustrative fragments, not complete runnable applications.

3. Object-level design patterns

Patterns are useful vocabulary, not proof of good design. In this section, say **problem -> shape -> example -> cost**. When an interviewer asks "Which pattern would you use?", clarify whether behavior varies, a vendor must be isolated, work is asynchronous, or state changes are constrained.

Strategy [High]

Plain English: Put each interchangeable rule behind the same small contract. Select the rule based on context.

Professional answer: "I use Strategy when one behavior has several valid policies, such as fee calculation by product. The caller uses one contract and each policy is tested separately. I avoid it for a single simple branch."

Shape: `FeePolicy.calculate(payment)`. Implementations could be domestic and international fee policies. Be careful that variation may depend on more than country; make selection rules explicit. **Cost:** More classes and a policy-selection mechanism.

Demonstration: A service must calculate different fees for domestic and cross-border instructions. It asks `FeePolicy.calculate(instruction)`; `DomesticFeePolicy` and `CrossBorderFeePolicy` implement that contract, and a `FeePolicySelector` chooses according to a documented rule.

Why Strategy, not State: The rule changes with instruction context; the payment does not transition from one policy object to another through its lifecycle. **Proof:** Run the same fee contract tests on both policies, plus selector tests at region/currency boundaries. **Avoid it:** A stable one-line conditional may be clearer.

Code anchor (illustrative fragment):

```
interface FeePolicy { Money fee(Payment p); }
final class DomesticFee implements FeePolicy {
    public Money fee(Payment p) { return Money.usd("1.50"); }
}
final class CrossBorderFee implements FeePolicy {
    public Money fee(Payment p) { return p.amount().times("0.01"); }
}
// A resolver chooses the rule; processor calls policy.fee(p).
```

Verify it: Change one fee rule without changing the processor. Test policy selection separately from fee computation.

Factory Method or simple factory [High]

Plain English: Put object creation and selection in one place. A simple factory can choose a `PaymentHandler` based on payment type. Factory Method specifically lets subclasses decide which concrete object to create.

Professional answer: "I separate complex creation from the business flow. If selection rules are centralized, a factory can return the correct handler while the processing code works with the handler contract. For a trivial object, direct construction is clearer."

Trap: Do not call every constructor wrapper a Factory Method. An interviewer may ask you to distinguish the formal pattern from a simple factory.

Demonstration: A `HandlerFactory.forRail(rail)` returns an `AchHandler` or `CardHandler`. The submission use case asks for a handler without knowing its constructor dependencies. This is a **simple factory**, because a selector method—not overriding subclasses—chooses the result.

Why a factory: Handler creation has real selection rules or nontrivial configuration. **Proof:** Unsupported rails receive an explicit rejection, and mapping tests show the correct handler for every supported rail. **Avoid it:** Construct `new FeeCalculator()` directly if no selection or construction complexity exists.

Code anchor (illustrative fragment):

```
final class PaymentHandlerFactory { // Simple factory.
    PaymentHandler create(PaymentRail rail) {
        return switch (rail) {
            case ACH -> new AchHandler();
            case CARD -> new CardHandler();
        };
    }
}
// Factory Method specifically delegates creation to subclasses.
```

Verify it: Verify the returned concrete handler for each supported rail; avoid a factory when one constructor is sufficient.

Builder [High]

Plain English: Construct a complex object step by step while making required data and validation clear.

Professional answer: "I use Builder when a request or configuration has many optional fields and construction should be readable. I validate required fields at build time and favor an immutable result."

Example: A payment instruction has mandatory tenant, amount, currency, and beneficiary, plus optional remittance details. **Cost:** Extra code; records or constructors may be enough for a small object.

Demonstration: A `PaymentInstruction` needs tenant, beneficiary, currency, and amount, with optional remittance text and delivery preference. A builder makes call sites readable: `builder.tenant(t).beneficiary(b).amount(a).currency(c).build()`. `build()` rejects missing mandatory fields and returns an immutable result.

Why Builder, not Factory: The question is *how to assemble and validate one complex value*, not which implementation to choose. **Proof:** Tests check missing required fields, invalid combinations, and

that the resulting value cannot be changed through shared mutable references. A short Java record or constructor may be better when there are only a few fields.

Code anchor (illustrative fragment):

```
PaymentInstruction instruction = PaymentInstruction.builder()
    .tenantId(tenantId)
    .amount(amount)
    .currency("USD")
    .beneficiary(beneficiary)
    .reference("invoice-42") // optional
    .build(); // validates required fields
```

Verify it: Omit each mandatory field in turn: build() must reject the incomplete value. Keep the built object immutable.

Adapter [High]

Plain English: Translate an external interface into the interface your application wants.

Professional answer: "I use Adapter to keep provider-specific APIs and data shapes outside business logic. Our core service calls a stable internal contract; the adapter handles provider request mapping, errors, and transport details."

Example: An old file-based payment interface and a new real-time service can implement the same internal `PaymentSubmissionPort` during migration. **Cost:** Translation logic and another layer; do not hide differences that change business semantics.

Demonstration: Provider A accepts `{customerRef, totalMinorUnits}` and returns status `QUEUED`; Provider B expects an XML payment file and returns a receipt code. Both are mapped to the internal `ClearingGateway.submit(Instruction)` contract and a carefully defined `SubmissionReceipt`.

Why Adapter: Translation isolates external syntax and transport from the core. **Proof:** Provider-specific contract tests cover units, currency, provider errors, and status mapping; core tests use a fake `ClearingGateway`. **Important:** If one provider's `QUEUED` means only receipt rather than confirmed execution, preserve that semantic difference rather than mapping it to `SETTLED`.

Code anchor (illustrative fragment):

```
interface ClearingGateway {
    GatewayResult submit(Payment p);
}
final class LegacyFileAdapter implements ClearingGateway {
    private final LegacyFileClient client;
    public GatewayResult submit(Payment p) {
        return mapResponse(client.writeLine(toLegacyLine(p)));
    }
}
```

Verify it: Test field and error mapping. The use case must not depend on the legacy file format.

Facade [High]

Plain English: Give callers one simple entry point to a complicated subsystem. The facade coordinates existing capabilities.

Professional answer: "I use a facade when clients should not orchestrate several low-level components themselves. For example, a user-onboarding facade can coordinate identity creation, role assignment, and audit recording behind one application operation."

Cost: A facade can become a very large service if it absorbs all business rules. Keep each collaborator's responsibility visible. A facade simplifies access; an adapter changes an interface.

Demonstration: New-user onboarding requires `IdentityCreator`, `RoleAssigner`, and `AuditRecorder`. Instead of requiring a controller to invoke them in the right order, expose `OnboardingFacade.createUser(request)` and let it coordinate their interfaces.

Why Facade: Callers need one business-level operation; no external interface needs translation. **Proof:** An application test verifies the call order and failure handling; each collaborator has its own tests. **Avoid it:** Do not hide every permission rule and persistence detail in a giant facade. In contrast, an Adapter primarily translates one interface into another.

Code anchor (illustrative fragment):

```
final class OnboardUserFacade {
    UserId onboard(NewUser input) {
        UserId id = identity.create(input);
        roles.assignDefaultRole(id);
        audit.recordUserCreated(id);
        return id;
    }
}
// One use-case entry point; collaborators keep their rules.
```

Verify it: Confirm that onboarding calls the collaborators in the required order and does not duplicate their domain rules.

Decorator [Medium]

Plain English: Wrap a component with another component that has the same contract and adds behavior.

Professional answer: "Decorator can add cross-cutting behavior such as metrics or caching around a repository or client without changing its core implementation. I use it when that behavior is composable and the wrapper's effect remains clear."

Example: `TimedPaymentGateway` measures a gateway call and delegates to `PaymentGateway`. **Cost:** Several wrappers can obscure call order and failure behavior. A cache wrapper needs a freshness policy.

Demonstration: `TimedGateway` implements `ClearingGateway`, contains another `ClearingGateway`, records elapsed time and outcome, then delegates `submit()`. A `TracingGateway` could wrap the same contract separately.

Why Decorator: The caller still sees the original contract, while behavior is composed around it. **Proof:** The delegate is called once, the return value/error behavior is preserved, and metrics are recorded on success and failure. **Cost:** Wrapper order matters. For example, timing outside a retry wrapper measures total retry duration, while timing inside measures each attempt.

Code anchor (illustrative fragment):

```java
final class TimedGateway implements ClearingGateway {
    private final ClearingGateway delegate;
    public GatewayResult submit(Payment p) {
        long start = clock.nanoTime();
        try { return delegate.submit(p); }
        finally { metrics.record(clock.nanoTime() - start); }
    }
}
```

Verify it: Measure both successful and failed calls. Verify that decoration does not change the gateway result.

Proxy [Medium]

Plain English: A proxy stands in front of another object and controls access to it. It may add transactions, security, lazy loading, or remote access.

Professional answer: "I expect proxy behavior at framework boundaries. In Spring, transaction advice commonly runs through a proxy, so a method calling another method on the same instance may bypass that advice. I test transaction behavior at the actual call boundary."

Difference from Decorator: Both wrap, but a proxy often controls access or lifecycle, whereas a decorator intentionally adds composable behavior. The structures can look similar; discuss intent and effects. See Spring transaction implementation.

Demonstration: `@Transactional` is placed on `PaymentWriter.commit()`, but another method on the *same instance* calls `this.commit()`. With normal Spring proxy-based transaction management, that internal call bypasses transactional interception. Call the transactional method through a separately injected Spring-managed bean (or design a correctly advised external use-case boundary).

Why Proxy: Spring intercepts calls that pass through the proxy; it is a runtime access boundary rather than a fee-rule extension. **Proof:** An integration test forces a database error and checks whether the expected writes roll back. Verify configuration and transaction manager behavior; annotations alone are not proof.

Code anchor (illustrative fragment):

```java
@Service
class PaymentService {
    void accept() { saveInTransaction(); } // Self-call bypasses proxy.
    @Transactional
    public void saveInTransaction() { /* DB writes */ }
}
// Prefer an external call through a proxied bean, or a
// correctly placed public transactional use-case boundary.
```

Verify it: Integration-test rollback through a real Spring proxy, not merely a direct method call. Official Spring docs confirm proxy self-invocation behavior.

Observer and publish/subscribe [Medium]

Plain English: One component announces an event and interested components react. In-process Observer means listeners in the same process. A message broker makes communication cross-process, with different durability and delivery concerns.

Professional answer: "I publish an event when other components need to react to a completed state change without being called synchronously. I distinguish a local callback from a durable event and plan for duplicates, ordering, and failed consumers."

Example: User-created event triggers audit or notifications. Do not send a durable event before its source transaction commits unless the design handles rollback.

Demonstration: After `UserCreated` is durably committed, an audit consumer records the change and a notification consumer sends a welcome message. Each subscribes to the fact without the user-creation use case having to call them synchronously.

Why pub/sub: Independent reactions can lag without blocking creation. **Proof:** Duplicate `UserCreated` delivery does not create two audit effects or duplicate customer-facing notifications (by consumer-side idempotency). **Boundary:** A local Java listener has a different durability guarantee than broker delivery; simply calling `publish()` before a DB commit can produce a false event if the transaction rolls back.

Code anchor (illustrative fragment):

```
record PaymentAccepted(PaymentId id, UUID eventId) {}
// After source state commits, publish a durable event.
subscriber.on(PaymentAccepted event) {
    if (dedup.insertIfAbsent(event.eventId()))
        projection.update(event.id());
}
// Dedup and side effect must be atomic in real storage.
```

Verify it: Deliver the same event twice: the projection must record one effect. This code is broker-neutral pseudocode for the callback body.

Command [Medium]

Plain English: Represent an operation as a value. This can separate receiving a request from executing, recording, retrying, or scheduling it.

Professional answer: "A command makes the requested action explicit, such as `ApprovePayment`. A handler validates and executes it. This can help with workflow, auditing, or async dispatch, but I do not add command objects to simple code without a need."

Trap: A command is an instruction or intent; an event states something that already happened. `ApprovePayment` and `PaymentApproved` are not interchangeable.

Demonstration: `ApprovePayment(paymentId, approverId, requestId)` describes a requested action. An `ApprovePaymentHandler` checks permissions and state, writes an approved transition, and may later emit `PaymentApproved(paymentId, approvedAt)`.

Why Command: The request can be validated, traced, queued, or retried as a distinct unit of work. **Proof:** Handling the same request identifier twice does not create duplicate approval effects; unauthorized commands are rejected. **Key distinction:** `ApprovePayment` can fail; `PaymentApproved` claims that approval already happened.

Code anchor (illustrative fragment):

```
record ApprovePayment(PaymentId id, UserId actor) {}
final class ApprovalHandler {
    void handle(ApprovePayment command) {
        Payment p = store.load(command.id());
        policy.requireAllowed(command.actor(), p);
        p.approve();
        store.save(p);
    }
}
```

Verify it: Run unauthorized and duplicate command tests. A command asks for work; it does not claim that work already happened.

State [High]

Plain English: Behavior changes depending on an object's state. Representing valid transitions prevents scattered status checks.

Professional answer: "I model payment status as explicit transitions and reject invalid changes. For a complex workflow, a State pattern or transition table can make the rules reviewable. For a small fixed workflow, an enum with a validated transition method may be simpler."

Example: `CREATED -> AUTHORIZED -> SETTLED`, with `FAILED` paths. Define whether reversal after settlement is a new compensating operation rather than simply moving status backward. **Cost:** More structure, especially if every status becomes a class.

Demonstration: An instruction has `CREATED`, `AUTHORIZED`, `SUBMITTED`, `SETTLED`, and `REJECTED` states. `settle()` is legal only after a confirmed provider outcome; `submit()` from `SETTLED` is invalid. Encode the transition table in a domain method or, if behavior is complex, separate State objects.

Why State, not Strategy: Behavior is constrained by the object's lifecycle, not a freely chosen policy. **Proof:** Test every permitted and forbidden state/action pair, including competing updates. **Avoid it:** Do not generate one class per enum value when a transition table expresses the rules clearly.

Code anchor (illustrative fragment):

```
enum Status { CREATED, AUTHORIZED, SETTLED }
boolean allowed(Status from, Status to) {
    return switch (from) {
        case CREATED -> to == Status.AUTHORIZED;
        case AUTHORIZED -> to == Status.SETTLED;
        case SETTLED -> false;
    };
}
```

Verify it: Test every allowed and forbidden edge. A fixed enum can be better than one State subclass per status.

Template Method [Medium]

Plain English: A base class fixes the overall algorithm and lets subclasses fill certain steps.

Professional answer: "Template Method fits a stable process with a few controlled variations. For example, each import may share validate, transform, persist, and report steps. I prefer composition if the variations combine in many ways or inheritance makes the hierarchy brittle."

Trap: A subclass must preserve the base process's contract. Hidden calls from the base class to overridable methods can complicate debugging.

Demonstration: `BaseStatementImporter.importFile()` fixes `validate -> parse -> persist -> summarize`. `CsvImporter` and `FixedWidthImporter` override only parsing; neither can bypass the shared validation in the base algorithm.

Why Template Method: The process order is stable, with a few controlled algorithm steps. **Proof:** Tests assert that invalid files never reach `persist()`, regardless of concrete importer. **Limit:** If validation, transformation, and persistence vary independently, composed strategies are easier to test and combine than a deep importer hierarchy.

Code anchor (illustrative fragment):

```
abstract class ImportJob {
    public final void run(File file) {
        validate(file);
        persist(transform(file));
        report();
    }
    protected abstract Rows transform(File file);
}
// Shared sequence is fixed; subclasses vary one step.
```

Verify it: A subclass may change transformation but not reorder validation, persistence, and reporting.

Chain of Responsibility [Medium]

Plain English: Pass a request through a sequence of handlers; each handler may process, reject, or pass it on.

Professional answer: "I use a chain when independent checks need an ordered pipeline, such as authentication, input limits, and audit enrichment. I document the order and whether a handler can stop processing."

Cost: Debugging becomes hard if order is implicit or failures disappear in the chain. Spring filters or interceptors are familiar examples, though their exact semantics differ.

Demonstration: An API request passes through `Authenticate -> TenantAuthorize -> ValidateSize -> RateLimit -> Controller`. Each step can allow the request to continue or reject it with an explicit error.

Why Chain: Multiple focused handlers must run in a known order. **Proof:** A test with an invalid tenant shows the request stops before the controller; logs identify which handler rejected it. **Risk:** Rate-limiting before authentication may require a different key and have different security properties; filter order is a design decision, not an accident.

Code anchor (illustrative fragment):

```java
interface Check {
    void verify(Request request);
}
final class ValidationPipeline {
    private final List<Check> checks;
    void run(Request request) {
        for (Check check : checks) check.verify(request);
    }
}
// Throwing from a check stops later checks.
```

Verify it: Test order and early exit. Security checks must not be accidentally skipped because of chain configuration.

Repository and Unit of Work [High]

Plain English: A Repository gives domain-oriented access to stored objects. A Unit of Work tracks related changes that should commit as one unit. In Spring/JPA, a persistence context and transaction often supply much of the Unit of Work behavior.

Professional answer: "I keep persistence queries behind meaningful repository operations and place the transaction around the business operation. I do not assume a repository method makes a multi-step workflow atomic; that depends on the transaction boundary and database guarantees."

Cost: A generic repository with dozens of vague methods can hide inefficient queries. A repository is not automatically an abstraction over every storage engine.

Demonstration: `ApprovePaymentUseCase` loads an instruction through `PaymentRepository`, changes its domain state, and appends an audit/outbox record inside **one local transaction**. A call to `save()` alone does not make the entire multi-step operation atomic.

Why Repository + Unit of Work: Repository hides storage-specific queries; a transaction coordinates the related state changes. **Proof:** Force an outbox insert failure and verify the approval update also rolls back. Add a concurrent-approval test with version checks or suitable locking. An HTTP call to an external provider is not automatically part of that transaction.

Code anchor (illustrative fragment):

```java
@Service
final class SettlementService {
    @Transactional
    public void settle(PaymentId id) {
        Payment p = repository.findRequired(id);
        p.markSettled();
        repository.save(p);
        outbox.save(PaymentSettled.of(p));
    }
}
// The local DB transaction covers both writes.
```

Verify it: Force an outbox insert failure: the payment update must roll back. A broker publish is not inside this local transaction.

Singleton: recognize it, use sparingly [Medium]

Plain English: A Singleton controls creation so there is one instance within a defined scope. In a distributed service, each process still has its own instance.

Professional answer: "I usually let the application container manage shared, stateless services. I do not use a global singleton for request state or payment deduplication because it cannot coordinate separate instances or survive restarts."

Trap: A Spring singleton bean is scoped to an application context. Shared mutable fields still need concurrency protection.

Demonstration: A stateless `FeeCatalog` can be a shared Spring bean. A mutable `processedIds = new HashSet<>()` stored in a singleton is **not** safe payment deduplication: concurrent requests race, each service instance has a separate set, and restarts erase it.

Why it matters: Instance scope is not a distributed consistency mechanism. **Proof:** Run the duplicate-request test across two app instances; expect a database uniqueness rule or other durable, coordinated store to decide the winner. Avoid storing per-request tenant or customer context in singleton fields.

Code anchor (illustrative fragment):

```
@Service // Spring singleton by default: per app context.
final class FeeCalculator {
    Money calculate(Payment payment) { /* stateless */ }
}
// BAD: a mutable field such as currentTenant
// would mix data from concurrent requests.
```

Verify it: Send concurrent requests for two tenants; a shared service must not retain tenant-specific mutable state.